Building an Adaptive and Cost-Efficient Production ML System: A Framework for Drift Monitoring and Automated Retraining

Final Report

Introduction

The objective of this project is to develop and evaluate a recommender system based on matrix factorization. While generating recommendations is important, analyzing the model's behavior and identifying ways to improve it are equally important. The system is based on TruncatedSVD and operates on implicit user-product interactions. Earlier stages of the project focused on model development, feature engineering, and optimization, while the final stage shifts toward testing, evaluation, and system reliability.

One of the main goals of this report is to present an accurate pipeline that can be easily transferred to the next engineer. It entails documenting the entire development process, identifying model weaknesses, and recommending ways to improve it. Although the model is used as a recommendation engine, the main aim of this project is to establish when it is appropriate to retrain the model based on users' new behavior. The observed performance degradation across temporal validation scenarios highlights the need for drift-aware, cost-efficient retraining strategies in production systems.

Methodology

The data consisted of user-product interaction data, with purchases treated as implicit indicators. Since the goal was to design a computationally viable system, a sample of users (2%) was selected for testing and analysis purposes. A time-based train-test split was applied, where past interactions were used for training and the most recent interactions for testing. This

approach ensures that the model is evaluated on unseen future behavior and better reflects real-world recommendation scenarios.

The recommendation model is built using TruncatedSVD with 10 latent components, selected based on prior tuning results. The model generates predicted scores for each user-product pair, and the top 10 highest-scoring products are recommended. Evaluation is based on Precision@10 and Recall@10, which measure how many relevant items appear in the top recommendations and how well the model covers user preferences. To improve interpretability, a Logistic Regression model was implemented using engineered features such as product interaction frequency, user reorder rate, and aisle-level activity, and LIME was used to explain individual predictions.

Reproducibility was ensured by using a fixed seed for randomness and by organizing the process into stages. This setup ensures reproducible experiments and enables meaningful comparisons between models of various architectures.

Results

The base model achieved Precision@10 and Recall@10 scores of 0.01966 and 0.01955, respectively. This shows that the model is able to identify only a small number of relevant items among the recommendations. As shown in Figure 1, model performance varies with the number of latent components. The results support selecting n_components = 10, which achieved the best balance between precision and recall.

	n_components	precision	recall
0	10	0.0190	0.016928
1	20	0.0148	0.013174
2	50	0.0132	0.010730

Figure 1. ***Model performance across different numbers of latent components.***

To further evaluate model stability, a temporal validation approach was used. In addition to being tested using the latest user behavior data, the model was also tested using the second-to-last order data to evaluate its performance across various time slices. In this situation, the Precision@10 and Recall@10 metrics were reduced to 0.0. This result shows that the model struggles to generalize across different temporal segments and may not be robust to changing user behavior.

Discussion

The results highlight several important insights about the model and its limitations in realistic scenarios. While the baseline performance provides a useful starting point, it is not sufficient for production use due to limited generalization across time. The model shows limited ability to generalize across time, as demonstrated by the temporal validation results.

The high zero-hit rate further underscores this limitation, as the model often fails to provide any correct recommendations for many users. This implies that the model does not effectively capture the user's preferences, particularly when behavior is more complex or less frequent. Furthermore, the unexpected performance trends from the different user history groups demonstrate that the model values simplicity or recency of user interactions over preference capture.

These findings suggest that the model is not well-equipped to handle dynamic user behavior, which is a known challenge in recommendation systems (Gama et al., 2014). Even more importantly, this behavior is tied directly to the necessity for adaptive systems that can determine when there is a deterioration in performance and then initiate training. This makes clear the necessity of developing systems that are both accurate and flexible.

Error Analysis

Error analysis revealed a high failure rate, with a zero-hit rate of 85.66%, meaning that a large proportion of users received no correct recommendations among the top 10 results. In total, 3,460 users were identified as zero-hit users, indicating that the model struggles to produce meaningful recommendations for a significant portion of the dataset. This highlights a major practical limitation.

As shown in Figure 2, users with shorter interaction histories achieved higher performance than those with longer histories. This unexpected pattern suggests that the model may rely more heavily on recent or simpler interaction patterns.

	history_group	precision@10	recall@10	hit_count
0	low_history	0.028302	0.023557	0.283019
1	medium_history	0.015315	0.013445	0.153153
2	high_history	0.006024	0.004658	0.060241

Figure 2. ***Recommendation performance by user history level (low, medium, high).***

The findings reveal two key issues. The first one is that the model struggles to adapt to changes in user behavior. The second problem is that the model fails to accommodate more experienced users.

Challenges and Solutions

During the development process, several practical challenges were encountered. One of the main issues was computational limitations when working with large-scale data, as initial experiments crashed sessions due to memory constraints. This problem was addressed by sampling a subset of users (2%) and optimizing intermediate data structures, enabling the pipeline to run efficiently while maintaining representative performance.

Another challenge was ensuring reproducibility while testing different configurations. This was resolved by introducing a fixed random seed and maintaining a consistent pipeline structure across all experiments. Additionally, a realistic evaluation strategy was needed, which required careful consideration, and rather than employing traditional cross-validation, as is common practice, a temporal validation approach was adopted to emulate conditions in the actual application environment better.

Future Work

Several improvements can be made to address the identified limitations. First, drift detection should be implemented to monitor changes in user behavior over time. This would allow the system to identify when the model becomes outdated and requires retraining.

Second, a cost-aware retraining strategy should be developed, where retraining decisions are based on performance thresholds rather than continuous updates. This approach reduces computational cost while maintaining model effectiveness and directly aligns with the broader goal of building efficient machine learning systems. Third, the system can be deployed to a production environment via API-based deployment, enabling real-time recommendations and integration with other systems.

Finally, more advanced models, such as neural collaborative filtering or hybrid recommendation systems, can be explored to improve performance and reduce failure rates. These approaches may better capture complex user behavior and provide more robust recommendations.

Conclusion

This project demonstrates the importance of going beyond model development and focusing on testing, evaluation, and system reliability. While the recommendation model provides baseline performance, it has clear limitations in terms of generalization and user-level accuracy. The use of temporal validation and error analysis provides a more realistic understanding of model behavior under changing conditions.

More importantly, the results highlight the need for adaptive systems that can respond to performance degradation over time. This supports the development of drift-aware and cost-efficient retraining strategies, which are critical for real-world machine learning applications. Overall, this project establishes a practical foundation for building adaptive and production-ready recommendation systems.

References

Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). Learning under concept drift: A review.

Koren, Y., Bell, R., & Volinsky, C. (2009). Matrix factorization techniques for recommender systems. Computer, 42(8), 30–37.